

Progetto Robotica - Lozio Francesco A.A. 2025/2026

1 Introduzione

Il progetto in esame è Micromouse, una competizione indetta da IEEE per risolvere labirinti 16x16 [1]. Sebbene la competizione prenda in esami robot e labirinti fisica, in questo progetto viene usato un simulatore che consente al robot (il "mouse") di muoversi entro un labirinto con muri fissati ma sconosciuti al robot.

I muri si presentano come taglio di connessioni tra una casella e l'altra: essi non occupano un'intera casella ma invece sono delle barriere poste a lato di una cella. Difatti, nella repository GitHub del progetto [2], viene indicata come una possibile rappresentazione per mappe sia quella di indicare per ogni casella la presenza del muro a Nord, Sud, Est e/o Ovest.

In questa relazione vengono presentati gli approcci visitati, con un particolare sulle strutture dati scelte per la memorizzazione della mappa che il mouse costruisce durante l'esplorazione. Vengono anche descritti algoritmi di ricerca classici per problemi di Path Finding.

2 Mappe

Sono state implementate 2 possibili mappe scelte in base allo stato dell'arte per questa competizione. Le mappe condividono comunque delle strutture adibite alla visualizzazione di micromouse e ad uso di algoritmi.

2.1 WallsMap

WallsMap tiene traccia dei muri rilevati durante l'esplorazione; quando si arriva ad una cella mai vista prima, si salva in un dizionario la direzione rilevata. In particolare, si usa *walls*, un dizionario la chiave è una tupla che contiene le coordinate della cella e la direzione mentre il valore è *True* se non c'è alcun ostacolo in quella direzione di quella cella, *False* altrimenti. Se invece la direzione non è stata controllata, *None*.

Questa è un'implementazione molto diretta: non ci sono convenzioni usate per facilitare la memorizzazione ed attribuzione di significato ai valori. Ciononostante, l'accesso ad uno specifico edge è fattibile in $O(1)$ grazie alla struttura a dizionario e la complessità spaziale è di $O(N)$ con N numero totale di celle.

2.1.1 Il riferimento al corso

WallsMap ricorda, per certi versi, la Occupancy Grid Map: mentre Grid Map tiene traccia di probabilità per una cella di essere occupata, WallsMap tiene traccia di se un muro è presente o meno. Inoltre, Grid Map consente di inserire una probabilità del 0.5 per quelle celle senza informazione cosa che in WallsMap viene fatto associando l'elemento python *None* al posto di *True* o *False*. La differenza sta però nel fatto che GridMap lavora a probabilità mentre, lavorando nel campo di Micromouse, WallsMap non ha bisogno di introdurre probabilità perché i sensori del mouse non sono probabilistici essendo dei controlli su una mappa nota a priori e i sensori controllano, data una cella, i muri presenti.

2.2 Bitmask Map

Standard de facto per rappresentazione di una mappa nella competizione micromouse [3]. Ogni cella viene rappresentata da un byte: i primi 4 bit (da destra) contengono informazioni sulla presenza di muri. In particolare, si segue la rappresentazione indicata:

- Nord: 0001 (1)
- Est: 0010 (2)
- Sud: 0100 (4)
- Ovest: 1000 (8)

L'uso di solo 4 bit lascerebbe spazio ad altri 4 bit di informazioni assegnabili ma qui non sono state usati *di fatto*.

L'implementazione fa uso di dizionari con tuple x, y per indicare la cella ed il valore è il byte con la rappresentazione sopra. In particolare, si è usato:

- *walls*: rappresenta i muri effettivamente presenti;
- *known*: rappresenta i muri rilevati dai sensori.

Tra i vantaggi dell'approccio, ci sono la memoria ridotta e la velocità delle operazioni: per controllare la presenza di un muro in una certa direzione, è sufficiente prendere i bit che rappresentano i muri e fare AND con la rappresentazione di quella direzione.

Bitmask consente ottenimento di dati in $O(1)$ ed ha una complessità spaziale di $O(N)$ con N numero di celle della mappa (cfr. per una 16x16 quindi avrà al massimo $16*16=256$ valori)

2.2.1 Il riferimento al corso

Nel corso veniva presentata una bitmap per cui veniva assegnato il valore 1 se veniva rilevato un ostacolo a distanza n, con n l'indice nell'array

- $[1][*][*][*] = 2^8$ possibilities over 16
- $[0][1][*][*] = 2^4$ possibilities over 16
- $[0][0][1][*] = 2^2$ possibilities over 16
- $[0][0][0][1] = 1$ possibility over 16

Figure 1: Slide del Corso, Proximity Sensor, Slide 13

Sebbene lo scopo fosse differente, la bitmap mostrata condivide la tecnica di trovare una rappresentazione sempre più compatta per indicare ostacoli. Nel caso del micromouse, la bitmap fa uso di singoli bit e convenzioni per rappresentare i muri nelle varie celle.

3 Algoritmi

Sono stati implementati 3 algoritmi standard per il Path Finding:

3.1 Breadth-First Search (BFS)

Il classico algoritmo di ricerca in ampiezza: si parte dalla cella di partenza e si prendono i vicini che minimizzano la distanza dal goal. Una volta scelto un vicino, si prende il prossimo vicino fino a quando non si raggiunge il goal.

3.2 Flood Fill

Flood Fill è un algoritmo classico per Path Finding nel micromouse: la scelta della prossima scelta avviene sulla base della sola euristica che considera la Manhattan distance tra la cella considerata ed il goal. Inoltre, ad ogni iterazione, le distanze delle celle vengono aggiornate per tenere traccia di nuovi muri scoperti.

3.3 A* Search

Classica ricerca di A* dove l'euristica usa la Manhattan Distance:

$$f(n) = g(n) + h(n) \quad (1)$$

$g(n)$ è invece la distanza dalla cella corrente fino alla casella desiderata.

4 Confronto Mappe

Di seguito le dimensioni delle varie strutture dati usate al raggiungimento del goal. Si considera la mappa standard da competizione, ossia 16x16. Viene usato l'algoritmo di Flood Fill poiché, per la mappa considerata, è quello che raggiunge il goal nel minor numero di passi. La dimensione delle strutture viene calcolata mediante `sys.getsizeof()`. La mappa presa in esame è l'esempio 4 fornito. La scelta di una sola mappa serve a sottolineare la differenza di peso in base al numero di celle visitate con algoritmi diversi.

Celle visitate con Flood Fill: 91.

Map Type	Dimensioni (B)
WallsMap	~ 94700
BitmaskMap	~ 47000

Per fare un paragone, viene eseguito lo stesso test con A*.
Celle visitate con A*: 144.

Map Type	Dimensioni (B)
WallsMap	~ 134430
BitmaskMap	~ 51020

L'incremento di dimensioni tra Wallsmap e Bitmask è differente: con circa 50 caselle visitate in più, Wallsmap aumenta di circa 40k B mentre Bitmask di appena 4k circa. Questo è anche dovuto ai diversi tipi di strutture: python introduce un overhead con strutture a dizionario per allocare spazio sulla base dei possibili valori: essendo che Wallsmap usa come chiavi tuple da 3 elementi, python deve riservare più spazio per Wallsmap rispetto a Bitmask.

Inoltre, i valori booleani in python hanno un peso di 28 byte, un valore decisamente superiore rispetto al singolo byte necessario per la Bitmask, confermando il suo primato nella competizione.

5 Confronto Algoritmi (Performance)

Vengono ora proposti confronti per le diverse tipologie di mappe fornite. Per semplicità, si assume che tutti gli algoritmi usino Bitmask come mappa.

Le Celle Attraversate differiscono da quelle visitate: una cella può essere attraversata più volte (cfr. backtracking) ma può essere visitata una volta sola.

Table 1: Mappa 1

Algoritmo	Celle visitate	Celle Attraversate	Numero rotazioni	Percentuale Esplorazione
BFS	121	142	60	47.3%
Flood Fill	121	142	60	47.3%
A*	108	118	54	42.2%

Table 2: Mappa 2

Algoritmo	Celle visitate	Celle Attraversate	Numero rotazioni	Percentuale Esplorazione
BFS	35	34	27	13.7%
Flood Fill	35	34	27	13.7%
A*	35	34	27	13.7%

Table 3: Mappa 3

Algoritmo	Celle visitate	Celle Attraversate	Numero rotazioni	Percentuale Esplorazione
BFS	106	120	59	41.4%
Flood Fill	106	120	59	41.4%
A*	132	176	93	52.6%

Table 4: Mappa 4

Algoritmo	Celle visitate	Celle Attraversate	Numero rotazioni	Percentuale Esplorazione
BFS	91	112	84	35.5%
Flood Fill	91	112	84	35.5%
A*	144	204	126	56.2%

Table 5: Mappa 5

Algoritmo	Celle visitate	Celle Attraversate	Numero rotazioni	Percentuale Esplorazione
BFS	204	306	216	79.7%
Flood Fill	204	306	216	79.7%
A*	118	134	90	46.1%

Come si evince dai risultati, Flood Fill e Breadth-First Search ottengono risultati equivalenti sulle mappe fornite mentre A* si differenzia: eccezion fatta per la mappa 2 che ha il percorso più breve, sulla mappa 1 e 5, A* attraversa meno (specialmente nella mappa 5 dove la differenza è di circa la metà) mentre per le mappe 3 e 4 A* performa peggio.

6 Conclusioni

Mentre per gli algoritmi la differenza consta principalmente nella struttura della mappa, con un solo esempio è possibile affermare i vantaggi della Bitmask Map, consolidandone il ruolo di stato dell'arte nella competizione Micromouse. Wallsmap si dimostra ugualmente utilizzabile ma non scala ugualmente bene come Bitmask Map.

Questo progetto aveva lo scopo di mostrare la differenza tra una mappa dall'implementazione più diretta (WallsMap) in contrapposizione con lo stato dell'arte in micromouse, ossia Bitmask Map. La necessità di ridurre queste dimensioni viene dal fatto che, nelle competizioni micromouse reali, il robot è spesso dotato di componenti più piccoli con poca memoria e quindi avere strutture dati efficienti consente al robot di lavorare correttamente.

La difficoltà nell'inserire mappe note nel corso risiede nel determinismo della simulazione: i sensori non sono affetti da probabilità e registrano un muro come visto nel momento in cui arrivano alla cella con il muro in questione. I "sensori" non fanno altro che leggere la mappa salvata all'inizio del cammino del file e poi salvare questi muri come effettivamente visitati con le relative informazioni. Una possibile effettivamente implementazione di mappe viste come una vera Occupancy Grid Map o RRT sarebbe fattibile se i sensori avessero, per esempio, una certa probabilità di sbagliare per simulare una distribuzione di probabilità.

References

- [1] Competizione Micromouse, Wikipedia (<https://en.wikipedia.org/wiki/Micromouse>).
- [2] Repository GitHub simulatore micromouse, GitHub (<https://github.com/mackorone/mms>).
- [3] Bitmask Map, <https://micromouseonline.com/micromouse-book/mazes-and-maze-solving/maze-files/>.